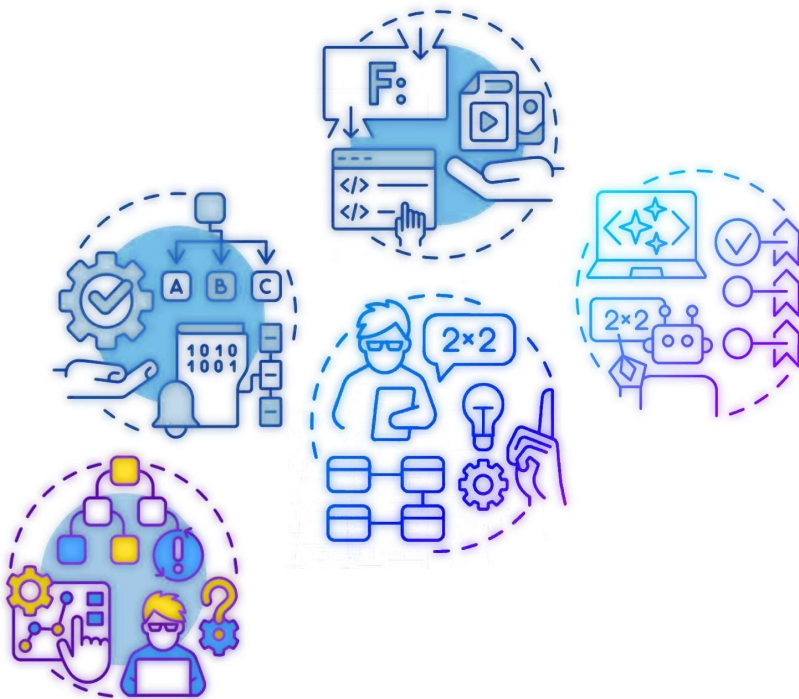




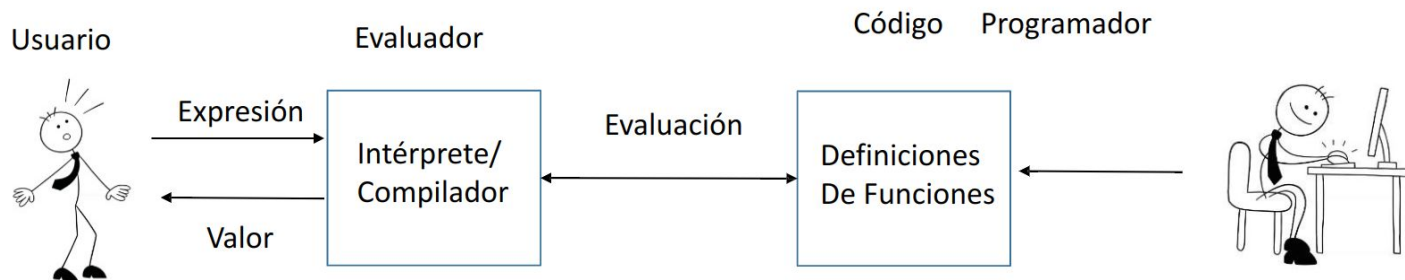
Paradigma Funcional

Esp. Ing. Viviana A. Santucci
AIA Federico Castoldi
Ing. V. Franco Matzkin
Ing. Cecilia Serafini

Tecnologías de la Programación

1 – Introducción

Es un paradigma declarativo que se basa en un modelo matemático de composición de funciones. En este modelo, el resultado de un cálculo es la entrada del siguiente, y así sucesivamente hasta que una composición produce el valor deseado, tal como sucede en la composición de funciones matemáticas.



1 – Introducción

nos enfocamos en el resultado y no en el
procedimiento

Expresan mejor **QUÉ** hay que calcular. **No especifican** tanto **CÓMO** realizar el mencionado cálculo a diferencia de lo que ocurre en los programas imperativos.

**LOS CÁLCULOS SE VEN COMO UNA FUNCIÓN
MATEMÁTICA QUE HACEN CORRESPONDER ENTRADAS Y
SALIDAS.**

1 – Introducción

No existe el concepto de celda de memoria que es asignada o modificada.

Existen valores intermedios que son el resultado de cálculos intermedios, los cuales resultan en valores útiles para los cálculos subsiguientes.

1 – Introducción

Todas las funciones tienen **transparencia referencial**. Esto es el resultado devuelto por una función sólo depende de los argumentos que se le pasen en esa llamada.

La repetición se modela utilizando la recursividad ya que no existe el valor de una variable.

2 – Historia

El modelo funcional de programación tiene sus orígenes en los años 30 con propuestas de Church y Shönfinkel y Curry. En los años 50, McCarthy diseñó el lenguaje LISP, seguido por Landin con la máquina abstracta SECD y los lenguajes ISWIM. En 1978, Backus promovió la programación funcional con su artículo y el diseño del lenguaje FP. Durante los años 70, Gordon trabajaba en el sistema LCF que incluía el lenguaje ML. En los años 80 surgieron muchos lenguajes funcionales y en 1987 se decidió diseñar Haskell como un lenguaje puramente funcional. Haskell unifica diversas características funcionales y presentaba nuevas soluciones como las clases de tipos. En 1998 se lanzó la versión estable Haskell98, continuando la investigación de nuevas extensiones al lenguaje.

3 – Características

Transparencia referencial: permite que el valor que devuelve una función está únicamente determinado por el valor de sus argumentos consiguiendo que una misma expresión tenga siempre el mismo valor. De esta manera, los valores resultantes son inmutables. No existe el concepto de cambio de estado.

Utilización de tipos de datos genéricos: permite aumentar su flexibilidad y realizar unidades de software genéricas, es una forma de implementar el polimorfismo en el paradigma funcional.

3 – Características

Recursividad: Basada en el principio matemático de inducción, que se ve expresada en el uso de tipos de datos recursivos, como las listas, y funciones recursivas que las operan.

Posibilidad de tratar a las funciones como datos mediante la definición de funciones de orden superior, que permiten un gran nivel de abstracción y generalidad en las soluciones. El uso correcto de las funciones de orden superior puede mejorar sustancialmente la estructura y la modularidad de los programas.

3 – Características

Los lenguajes funcionales **relevan al programa de la compleja tarea de gestión de memoria**. El almacenamiento se asigna y se inicializa implícitamente, y es recuperado automáticamente por un recolector de la basura (garbage collector). La tecnología para la asignación de memoria y de la recolección de la basura están actualmente bien desarrollados por lo que su costo de funcionamiento es leve.

no existen los conceptos de variables o el concepto de cambio de estado.

La manera de construir abstracciones es a través de funciones.

Basado en la matemática y en la teoría de funciones

4 – Ventajas y limitaciones

Ventajas

- 1) Fácil de formular matemáticamente.
- 2) Administración automática de la memoria.
- 3) Simplicidad en el código.
- 4) Rapidez en la codificación de los programas.

4 – Ventajas y limitaciones

Limitaciones

- 1) No es fácilmente escalable.
- 2) Difícil de integrar con otras aplicaciones.
- 3) No es recomendable para modelar lógica de negocios o para realizar tareas transaccionales.

5 – Áreas de aplicación

- 1) Demostraciones de teoremas: Por su naturaleza “funcional” este paradigma es útil en la demostración automática de teoremas, ya que permite especificar de manera adecuada y precisa problemas matemáticos.
- 2) Creación de compiladores, analizadores sintácticos: Su naturaleza inherentemente recursiva le permite modelar adecuadamente estos problemas.
- 3) Resolver problemas que requieran demostraciones por inducciones.
- 4) En la industria se puede usar para resolver problemas matemáticos complejos.
- 5) Se utiliza en centros de investigaciones y en universidades.

6 – Lenguajes en el paradigma funcional

Los lenguajes de programación funcionales se originan en el Lambda Cálculo, que es fundamental en su desarrollo, como se ve en lenguajes como Lisp, Hope, Miranda, Haskell, ML y Scheme. El Lambda Cálculo ofrece una estructura básica para la creación de primitivas de programación y se basa en funciones matemáticas para procesar argumentos y generar resultados.

Lisp, creado por John McCarthy, fue el pionero de los lenguajes funcionales y mantuvo su singularidad durante años. Sin embargo, con la complejidad creciente de los programas informáticos, surgió la necesidad de una verificación más rigurosa por parte de las computadoras, lo que llevó a la aparición de lenguajes funcionales tipados en los años ochenta, como ML, Scheme (derivado de Lisp), Hope y Miranda.

6 – Lenguajes en el paradigma funcional

ML y Scheme son particularmente populares, aunque han realizado ciertas adaptaciones hacia los lenguajes imperativos.

Un grupo de investigadores destacados desarrolló Haskell, un lenguaje que combina las mejores características de los lenguajes existentes, siendo un lenguaje puramente funcional y de propósito general.

6 – Lenguajes en el paradigma funcional

Racket y Scheme comparten muchas similitudes, pero también tienen diferencias significativas:

Origen: **Racket es un derivado de Scheme.** Originalmente se llamaba PLT Scheme

Estándares: Scheme está definido por un estándar, mientras que Racket no lo está. Sin embargo, **Racket puede ejecutar código estándar de Scheme.**

Inmutabilidad: **Las listas en Racket son inmutables** por defecto, mientras que las listas en Scheme son mutables.

6 – Lenguajes en el paradigma funcional

Bibliotecas: Racket incluye una gran cantidad de bibliotecas estándar (por ejemplo, servidor web) que otros esquemas no tienen.

Compilación: Racket es un compilador. Compila el código de Racket a bytecode, y luego compila el bytecode a código de máquina con un compilador JIT (Just-In-Time).

Racket no es un superconjunto ni un subconjunto de Scheme.

8 – Lenguajes en el paradigma funcional

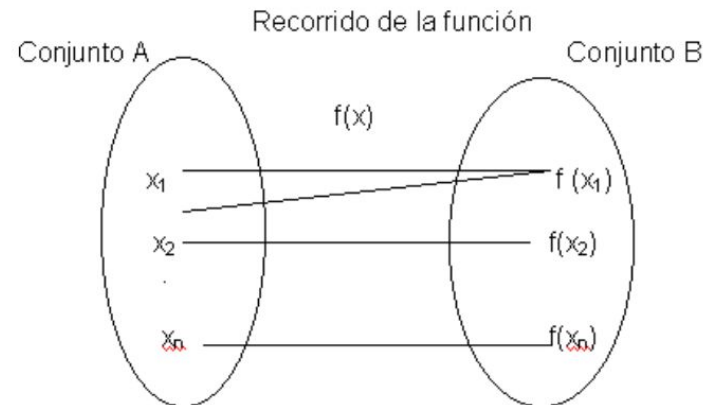
Bibliotecas: Racket incluye una gran cantidad de bibliotecas estándar (por ejemplo, servidor web) que otros esquemas no tienen.

Compilación: Racket es un compilador. Compila el código de Racket a bytecode, y luego compila el bytecode a código de máquina con un compilador JIT.

Racket no es un superconjunto ni un subconjunto de Scheme.

9 – Conceptos generales - Función

Una **función** es una regla de asociación que relaciona dos o más conjuntos entre si; generalmente cuando tenemos la asociación de dos conjuntos la función se define como una regla de asociación entre un conjunto llamado dominio con uno llamado codominio o imagen.



9 – Conceptos generales - Función

Una función **f** entre dos **conjuntos A y B**, es una correspondencia que a cada elemento de un subconjunto de **A**, llamado “**Dominio de f**”, le hace corresponder uno y sólo uno de un subconjunto **B** llamado “**Imagen de f**”.

$$f: A \rightarrow B$$

f es una función de A en B, o f es una función que toma elementos del dominio A y los aplica sobre otro llamado imagen B.

9 – Conceptos generales - Función

Dominio de una función: son todos los valores que puede tomar el conjunto del dominio y que encuentra correspondencia en el conjunto llamado imagen. Cuando se habla del plano, el dominio es el intervalo de valores que están sobre el eje de las X's y que nos generan una asociación en el eje de las Y's.

Codominio o imagen de la función: este conjunto está formado por los valores que puede tomar la función; en el caso del plano son todos los valores que puede tomar la función o valores en el eje de las Y's.

9 – Conceptos generales - Función

Las funciones se expresan con la siguiente sintaxis:

$$f(x) = 2 * x$$

donde:

x: argumento nominal que representa a los elementos del dominio

f: nombre de la función

En el lado derecho se representa el elemento del codominio o imagen o la forma de obtenerlo.

9 – Conceptos generales - Función

para calcular la **imagen de x a partir de f**, deben usarse otras funciones, las cuales también deberán definirse, por ejemplo.

$$f(x) = 2 * x$$

$$g(x) = f(x) + 4$$

donde **f**, **g**, *****, **+** son funciones

9 – Conceptos generales - Función

para calcular la **imagen de x a partir de f**, deben usarse otras funciones, las cuales también deberán definirse, por ejemplo.

$$f(x) = 2 * x$$

$$g(x) = f(x) + 4$$

donde **f**, **g**, *****, **+** son funciones

9 – Conceptos generales - Función

1. Las funciones establecen relaciones entre dos conjuntos, imponiendo la restricción única de que un elemento no puede tener dos imágenes,
2. La definición de funciones es una estructura jerárquica, en la cual las funciones más simples aparecen en la definición de las funciones más complejas.
3. En las definiciones, las funciones utilizadas a la derecha del signo de igualdad se referencian solo por su nombre. El significado puede obtenerse de reemplazar el nombre por su definición.

9 – Conceptos generales - Funciones

es aquella función que al ser derivada nos da como resultado la función original $f(x)$. Por ejemplo, si $f(x) = 2x$, entonces una función primitiva de $f(x)$ sería $F(x) = x^2$, ya que la derivada de x^2 es $2x$.

4. Esta sucesión se clausura con **funciones primitivas**
5. Básicamente se aplican los conceptos del diseño modular, en el cual un problema complejo se descompone en problemas más simples.
6. Cada uno de ellos es un problema primitivo, o bien, deberá volver a descomponerse.
7. La facilidad de referenciar una función por su nombre, permite que la misma sea empleada sin tener la necesidad de conocer su estructura interna.

9 – Conceptos generales - Función

8. Esta estrategia permite diseñar realizando refinamientos sucesivos.

9. La estructura de un módulo complejo puede definirse, sin que necesariamente estén definidos los módulos componentes.

9 – Conceptos generales - Función - Sintaxis en Funcional

Las funciones se denominan expresiones y se pueden representar con la siguiente sintaxis:

$\langle \text{expresión} \rangle ::=$

$\langle \text{variable} \rangle \mid$
 $\langle \text{constante} \rangle \mid$
 $(\langle \text{variable} \rangle, \dots, \langle \text{variable} \rangle)$
 $\langle \text{expresión} \rangle \mid \langle \text{expresión} \rangle$
 $(\langle \text{expresión} \rangle, \dots, \langle \text{expresión} \rangle)$

9 – Conceptos generales - Función - Sintaxis en Funcional

Todas las expresiones denotan un valor, pueden ser expresiones que no admiten ser aplicadas a ningún argumento, ya que están totalmente evaluadas o expresiones que pueden ser aplicadas a un cierto número de argumentos para ser totalmente evaluadas.

Las formas permitidas por la gramática son:

A. **expresiones atómicas**, que pueden ser `<variable>` y `<constante>`

B. **expresiones compuestas**

abstracciones funcionales: `(<variable>, . . . , <variable>) <expresión>`

aplicaciones funcionales: `<expresión> (<expresión>, .... , <expresión>)`

9 – Conceptos generales - Abstracción funcional

Una **abstracción funcional** es una **expresión por la cual se define una función**. Considérese la siguiente expresión:

$$3*5$$

mediante una abstracción funcional puede construirse una expresión de tal forma que el producto anterior sea un caso particular de ésta.

9 – Conceptos generales - Abstracción funcional

Una **abstracción funcional** es una **expresión por la cual se define una función**. Considérese la siguiente expresión:

$$3*5$$

mediante una abstracción funcional puede construirse una expresión de tal forma que el producto anterior sea un caso particular de ésta.

9 – Conceptos generales - Abstracción funcional

Por ejemplo:

$$(x) \ 3 * x$$

es una forma más general, y mediante una aplicación de la misma puede obtenerse la expresión original.

9 – Conceptos generales - Abstracción funcional

También se podría generalizar más la abstracción funcional, expresando:

$$(x, y) \ x * y$$

donde la definición de la abstracción funcional cumple con la construcción general:

$$(<\text{variable}>, \dots, <\text{variable}>) \ <\text{expresión}>$$

También se puede expresar de la siguiente forma:

$$(x) (y) \ <\text{expresión}>$$

tanto $(y) \ <\text{expresión}>$ como $(x) (y) \ <\text{expresión}>$ son abstracciones funcionales. Esta opción indicaría que la imagen será una función de y .

9 – Conceptos generales - Func. Orden Superior

Es una función tal que alguno de sus argumentos es una función o que devuelve una función como resultado.

- Son útiles porque permiten capturar esquemas de cómputo generales (abstracción).
- Son más útiles que las funciones normales (parte del comportamiento se especifica al usarlas).

9 – Conceptos generales - Func. Orden Superior

Por ejemplo:

Definimos una función que invoque a otra función dos veces, por ejemplo el incremento en 1. Para expresar esta función colocamos en el primer argumento, en lugar de una variable, una función llamada f , quedando la siguiente expresión:

$$(f, x) \rightarrow f(f(x)) \text{ ó } (f) \rightarrow (x) \rightarrow f(f(x))$$

9 – Conceptos generales - Func. Orden Superior

donde:

- f representa la expresión o función.
- x es una variable.
- $(x) f (f (x))$ es una abstracción funcional (que posee la variable ' x ' y la expresión ' f ').
- $f (f (x))$ es una abstracción funcional (define la expresión ' f ').
- $f (x)$ es una aplicación funcional (define el cuerpo de la abstracción funcional anterior).

9 – Conceptos generales - Func. Orden Superior

Esta expresión indicaría que, la imagen de la función sobre la variable f es una función sobre la variable x que aplica dos veces f a x

Si asociamos un nombre a esta función:

$\text{dos_veces} = (f) (x) f (f x)$

Donde:

- $(x) f(f(x))$ es la expresión
- $y (f) (x) f(f(x))$ la abstracción funcional

9 – Conceptos generales - Func. Orden Superior

Si consideramos la función `inc` (que incrementa en 1 un valor), por ejemplo:

$$\text{inc} = (\lambda x) x+1$$

Si asociamos `f` con la función `inc`, la imagen de `inc` de acuerdo a `dos_veces` sería:

$$\text{dos_veces} (\text{inc}) ==> (\lambda x) \text{inc}(\text{inc}(x))$$

la aplicación de `dos_veces` a una función da como resultado otra función, la cual obviamente puede aplicarse a su vez a otro valor.

9 – Conceptos generales - Func. Orden Superior

Si probamos con el valor de $x = 3$:

`(dos_veces (inc)) 3 ==> (3) inc( inc(3) )`

4

5

En este ejemplo podemos destacar que los argumentos y resultados de la aplicación de funciones pueden ser a su vez funciones, las cuales pueden ser también aplicadas a otros argumentos.

Este tipo de función es denominado **FUNCIÓN DE ORDEN SUPERIOR** denotando el hecho que sus argumentos y resultados pueden ser funciones.